VELLORE INSTITUTE OF TECHNOLOGY

DATABASE PROJECT REPORT

Rankstack: Contest Platform with Live Leaderboard

Review 3 – Final Project Submission and Demonstration

Submitted in partial fulfillment of the requirements for
BCSE302P – Database Systems Lab

Student Names:	Upayan Mazumder (24BDS0367) Aditya Rawat (24BCE2992) Shloak Sinha (24BDS0378)
Course Code & Title:	BCSE302P – Database Systems Lab
Repository URL:	github.com/upayanmazumder/rankstack
Live Deployment:	rankstack.upayan.dev

Academic Year 2026 – Fall Semester

Abstract

Competitive programming platforms handle sudden traffic spikes during live contests. Relational databases experience high CPU load and connection pool exhaustion when hundreds of participants submit code and poll rankings simultaneously. Frequent grouping and sorting operations degrade response times and increase server resource utilization under concurrent workloads.

This project implements Rankstack, an open source contest evaluation platform with live leaderboards. The database architecture separates persistent storage from operational caching. MongoDB 7.0 acts as the system of record. It stores five primary collections (users, teams, contests, problems, submissions) plus one coordination collection (dirty_leaderboards). Server-side validation schemas enforce structural integrity while supporting polymorphic problem types. Multi-document transactions run on a single-node replica set to preserve atomic consistency across submissions, problem statistics, and user scores. Redis 7.4 manages transient state. It uses sorted sets for contest leaderboards, string keys with time-to-live expiration for authenticated user sessions, and fixed-window rate limiting for submission throttling. An atomic version marker coordinates incremental score deltas with full leaderboard rebuilds.

The implementation was evaluated using a seeded dataset of 117 documents across six collections. Execution plans confirmed index-backed query execution using index scans that avoid collection scans. Redis sorted sets returned participant rankings through a lock-light read path, preventing stale-delta overwrites during rebuilds. Multi-document transactions prevented partial state updates during simulated write failures. Analytical aggregation pipelines computed user metrics, team standings, and contest summaries alongside persistent transactional records.

The hybrid MongoDB and Redis architecture delivers consistent data persistence, low-latency leaderboard retrieval, and coordinated cache state. Serving live rankings from in-memory sorted sets isolates continuous polling from the primary database during high-volume contests.

Keywords: MongoDB, Redis, Sorted Sets, Live Leaderboard, Multi-Document Transactions, Polymorphic Schema, Competitive Programming.

Contents

Chapter 1: Introduction 4
1.1 Background of the Study 4
1.2 Motivation 4
1.3 Problem Statement 4
1.4 Aim and Objectives 4
1.5 Scope of the Project 4
1.6 Limitations 5
1.7 Methodology Overview 5
Chapter 2: Literature Review 5
2.1 Overview of the Application Domain and Its Data Needs 5
2.2 Review of Data Models 5
2.3 Review of Existing Systems and Related Work 6
2.4 Comparison of DBMS Platforms and Tools 6
2.5 Standards and Best Practices 6
2.6 Gap Identified 6
2.7 How This Project Addresses the Gap 6
2.8 Summary of the Chapter 7
Chapter 3: System Analysis and Requirements 7
3.1 Existing System and Its Drawbacks 7
3.2 Proposed System and Its Advantages 7
3.3 Feasibility Study 7
3.4 Users, Roles and Access Needs 7
3.5 Requirements Specification 8
3.6 Business Rules and Constraints 8
Chapter 4: Database Design 9
4.1 Design Approach and Principles 9
4.2 System Architecture Diagram 9
4.3 Data Flow Diagram 9
4.4 Conceptual Design: ER / EER Diagram 10
4.5 Logical Design 11
4.6 Normalization and Consistency Analysis 12
4.7 Physical Design 13
4.8 Integrity Constraints 13
Chapter 5: Implementation 14
5.1 Development Environment and Setup 14
5.2 DBMS, Tools and Libraries Used 14
5.5 Views and Indexes Created 14
5.7 Application Connectivity 14
5.8 Version Control and Project Repository 14
5.9 Challenges Faced and Solutions Adopted 14
Chapter 6: Query Design and Performance Tuning 15
6.1 Representative Queries 15
Chapter 7: Results and Discussion 17
7.1 Output Screenshots and Sample Reports 17
7.2 Evaluation Against Objectives 19
7.3 Comparison with Existing Systems 20
7.4 Discussion of Findings 20
7.5 Limitations Observed 20
Chapter 8: Conclusion and Future Work 20
8.1 Summary of the Work 20
8.2 Key Contributions 20
8.3 Future Enhancements 20
8.4 Concluding Remarks 21
Part C: References 21
Appendices 22
Appendix A: Complete DDL Scripts 22
Appendix B: Sample Data and DML Scripts 24
Appendix C: Stored Procedures, Functions and Triggers (Full Code) 24
Appendix D: Complete Query Set with Outputs and Verification 26
Appendix E: Full-Size ER Diagram and Relational Schema 28
Appendix F: Application Screenshots 28
Appendix G: User Manual 28

Chapter 1: Introduction

1.1 Background of the Study

Competitive programming platforms host timed coding contests where hundreds of participants submit code solutions concurrently. The platform must ingest submissions, evaluate test cases, calculate scores, and update leaderboards in real time. Workloads exhibit sharp bursts: submission volume peaks during the opening and closing minutes of a contest.

1.2 Motivation

Traditional database systems rely on relational tables for contest storage. Relational engines enforce rigid tabular schemas and execute aggregation queries with sorting to compute standings. Under continuous client polling, repeated execution of grouping queries saturates database CPU capacity and exhausts connection pools. Stonebraker et al. demonstrated that specialised operational stores outperform monolithic engines for specific access patterns [8]. Decoupling durable records into document storage and serving live rankings from in-memory sorted sets resolves this performance bottleneck [5].

1.3 Problem Statement

Synchronizing high-frequency code submissions with real-time leaderboard calculations introduces specific technical challenges:

1. High database CPU consumption: Executing repeated GROUP BY and ORDER BY queries across thousands of submission rows starves database compute resources.
2. Connection pool starvation: N polling clients multiplied by continuous request rates consumes connection-pool capacity.
3. Rigid schemas: Relational tables struggle to represent multiple choice, coding, and subjective problem types without sparse, nullable columns.
4. Partial state anomalies: Unhandled failures during submission insertion leave attempt counters and user scores desynchronized.
5. Cache synchronization races: Incremental score deltas applied to an in-memory cache during a full cache rebuild risk being overwritten or applied twice.

1.4 Aim and Objectives

Aim: Design, implement, and evaluate Rankstack, a contest platform utilizing MongoDB 7.0 for persistent storage and Redis 7.4 for operational caching.

Specific Objectives:

1. Design a two-tier database architecture decoupling five primary collections plus one coordination collection from in-memory caching layers.
2. Enforce schema integrity and polymorphic problem modeling using MongoDB server-side `$jsonSchema` validation.
3. Maintain multi-document ACID transactions across submission creation, attempt counter updates, and score increments.
4. Deliver live contest leaderboards through a lock-light read path using Redis sorted sets and atomic version coordination.
5. Implement five analytical aggregation pipelines for leaderboard computation, difficulty metrics, and team standings.

1.5 Scope of the Project

The scope encompasses:

- Persistent layer: MongoDB 7.0 single-node replica set (`rs0`) for durable document storage and ACID transactions.
- Cache layer: Redis 7.4 for sorted-set leaderboards, session TTLs, and fixed-window rate limiting.
- Backend API: Python 3.14 with FastAPI providing REST controllers and transaction services [6].
- Web Client: Next.js 16 (App Router) with React 19, TypeScript, and Tailwind CSS v4.
- Infrastructure: Docker Compose orchestration and automated database seeding (`117` documents).

Code evaluation runs through deterministic in-process evaluation handlers. Isolated distributed execution sandboxes (such as gVisor or nsjail) are deferred to future enhancements.

1.6 Limitations

1. Single-node replica set: The development environment runs a single-node replica set (`rs0`). It provides transactional semantics but lacks automated multi-host failover. Production deployment targets 99.9% availability using a three-node replica set.
2. In-process code evaluation: Code evaluation runs within the backend application process. While adequate for deterministic laboratory workloads, untrusted production code execution requires containerized execution sandboxes.
3. Language support: Prototype evaluation supports Python, JavaScript, and direct text matching for multiple choice questions.

1.7 Methodology Overview

The project followed the Database Life Cycle (DBLC) [2]:

1. Requirements Analysis: Identify submission rates, read-to-write ratios, and polymorphic question attributes.
2. Conceptual Design: Develop an Entity-Relationship (ER) model representing entities, cardinalities, participation constraints, and problem specialization.
3. Logical Design: Map entities into six BSON collections (users, teams, contests, problems, submissions, dirty_leaderboards) with `$jsonSchema` validation.
4. Physical Design: Configure WiredTiger B-tree indexes, compound index prefix orders, and Redis sorted sets.
5. Implementation: Write FastAPI database dependencies, PyMongo client session transactions, and Redis Lua scripts.
6. Testing: Audit index usage via `.explain("executionStats")`, test transaction atomicity under simulated write failures, and benchmark endpoint latencies.
7. Maintenance: Configure automatic TTL expiration for ended contest leaderboards (`expire_leaderboard`, 7-day TTL) and inactive sessions.

Chapter 2: Literature Review

2.1 Overview of the Application Domain and Its Data Needs

Contest platforms exhibit asymmetric read and write traffic. Leaderboards are polled continuously by active contestants, while submissions arrive in unpredictable spikes. Problem specifications are heterogeneous: multiple choice questions require option arrays and answer keys; coding problems require input formats, constraint strings, and test case arrays; subjective problems require word limits and grading rubrics.

2.2 Review of Data Models

1. Relational Model (PostgreSQL, MySQL): Relational engines provide mature ACID transactions and multi-version concurrency control (MVCC) [1], [9]. However, computing contest leaderboards requires scanning submission rows, aggregating scores, and performing sorting operations (`ORDER BY total_score DESC`). Under high polling concurrency, these repeated aggregation queries consume extensive CPU capacity. Furthermore, modeling polymorphic problem structures requires table-per-hierarchy (causing sparse nullable columns) or table-per-type (requiring multi-table joins).
2. Document Model (MongoDB): Document databases store self-contained BSON documents [3]. Arrays and nested objects allow bounded relationships to be embedded directly within parent documents. Server-side `$jsonSchema` validation enforces schema constraints while permitting polymorphic subtypes through the `oneOf` operator. Running MongoDB as a replica set provides multi-document ACID transactions with document-level concurrency control via WiredTiger [4].
3. Key-Value Model (Redis): In-memory key-value stores provide sub-millisecond execution times [5]. To achieve high read availability, in-memory caches accept distributed partition trade-offs described by Gilbert and Lynch [10]. Redis sorted sets (`ZSET`) maintain unique members ordered by floating-point scores using a dual skip-list and hash-table structure [5]. Score updates (`ZINCRBY`) and rank range queries (`ZREVRANGE`) execute in `O(log N)` time, making Redis suited for live leaderboards.
4. Column-Family Model (Apache Cassandra): Columnar stores provide distributed write throughput across partitioned clusters [12]. While Cassandra supports single-partition lightweight transactions via Paxos, it

lacks multi-document ACID transactions and ad-hoc analytical aggregation pipelines, making it ill-suited for contest state management.
5. Graph Model (Neo4j): Graph engines optimize deep path traversals. Contest platforms follow hierarchical and tabular data relationships, making graph traversal unnecessary.

2.3 Review of Existing Systems and Related Work

- Codeforces: Based on public platform overviews, Codeforces pairs custom execution invokers with relational database clusters. Concurrency is managed through aggressive page-level caching and scheduled batch recalculation of standings.
- LeetCode: Public engineering articles describe a multi-tier architecture using relational storage (MySQL) combined with distributed Redis caching tiers to serve problem catalogs and contest rankings to participants.
- Kattis and DOMjudge: Academic contest platforms designed for ICPC competitions, typically relying on relational backends with deterministic sequential submission processing.

2.4 Comparison of DBMS Platforms and Tools

Feature	MongoDB 7	PostgreSQL 16	Redis 7	Cassandra 4
Data Model	Document (BSON)	Relational	Key-Value In-Memory	Wide-Column
Schema Flexibility	Dynamic + `$jsonSchema`	Rigid DDL	Schema-free	Rigid Column Family
Leaderboard Query Speed	O(N log N) scan	O(N log N) sort	O(log N + M) instant	Unoptimized scan
Multi-Document ACID	Yes (Replica Set)	Yes (Native)	No (Atomic Multi/Exec)	Single-partition only
Complex Aggregations	Native Pipeline	SQL Group By / Window	Limited	Basic CQL Group By
Primary Fit in Rankstack	System of Record	Alternative Relational	Fast-Path Cache	Unsuitable

Architectural Justification: MongoDB vs. PostgreSQL with JSONB

A viable alternative is PostgreSQL with `JSONB` columns combined with Redis. While PostgreSQL supports JSON storage and extensions like Citus support distributed tables, MongoDB was selected for four engineering reasons:

1. Native BSON Type Fidelity: MongoDB preserves distinct 64-bit integers, double-precision floats, UTC `ISODate` instances, and 12-byte `ObjectId` primary keys natively without string serialization overhead.
2. Polymorphic Document Validation: MongoDB `$jsonSchema` validates polymorphic structures at the engine level using the `oneOf` operator, verifying required subtype fields while rejecting malformed documents before disk write.
3. Uniform Aggregation Pipelines: Analytical queries (e.g. multi-stage `$lookup`, `$unwind`, and `$group`) execute through a uniform declarative JSON syntax without bridging SQL and relational-to-JSON operators.
4. Native Sharding Path: MongoDB provides horizontal sharding based on contest partitions (`{ contestId: 1, submittedAt: 1 }`), whereas sharding relational JSONB tables requires external orchestration.

2.5 Standards and Best Practices

- BSON document limits: Keep documents well below the 16 MB limit by embedding only bounded arrays [3].
- Index selectivity: Follow the Equality, Sort, Range (ESR) rule when designing compound indexes.
- Caching coordination: Coordinate cache writes with database commits using versioned revision markers to prevent stale score overwrite bugs [11].
- TTL eviction: Enforce time-to-live policies on operational keys to automate memory reclamation.

2.6 Gap Identified

Existing small-scale platforms either execute repeated SQL queries that saturate database CPU capacity during active contests, or employ naive cache invalidation strategies that introduce race conditions between score updates and background cache rebuilds.

2.7 How This Project Addresses the Gap

Rankstack decouples persistence from live operational querying. MongoDB transactions guarantee data consistency across collections [4], while Redis sorted sets serve real-time rankings [5]. An atomic version marker (`leaderboard_revision`) coordinates score deltas with cache rebuilds, ensuring that score updates never corrupt an unbuilt or rebuilding cache.

2.8 Summary of the Chapter

The literature confirms that combining a document database for durable records with an in-memory key-value cache for operational rankings delivers write safety and low read latency without database connection saturation.

Chapter 3: System Analysis and Requirements

3.1 Existing System and Its Drawbacks

Traditional academic contest platforms use monolithic relational architectures. Standings are computed by executing SQL queries on every participant request:

```sql
SELECT user_id, SUM(score) AS total_score
FROM submissions
WHERE contest_id = ?
GROUP BY user_id
ORDER BY total_score DESC;
```

Drawbacks:

1. High CPU Utilization: Under concurrent polling, repeated sorting and grouping operations consume significant server CPU time.
2. Connection Exhaustion: Hundreds of polling clients quickly saturate available database connections.
3. Concurrency Bottlenecks: While relational MVCC ensures readers do not block writers [1], intense submission writes compete with repeated analytical aggregation queries for CPU and disk I/O bandwidth.
4. Rigid Schemas: Introducing new problem formats requires DDL table alterations or sparse nullable columns.

3.2 Proposed System and Its Advantages

1. In-Memory Sorted Sets: Redis `ZREVRANGE` retrieves top participants through a lock-light read path [5].
2. Transactional Integrity: MongoDB multi-document transactions ensure atomic updates across submissions, counters, and participant scores [4].
3. Polymorphic Validation: Server-side `$jsonSchema` validates multiple choice, coding, and subjective problems within a single collection using `oneOf` blocks.
4. Ephemeral Auth State: Opaque session tokens and submission rate limiters reside in Redis with automatic TTL expiration.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The platform uses production-ready, open-source technologies: MongoDB 7.0, Redis 7.4, FastAPI, and Next.js 16. Stable drivers (`pymongo`, `redis-py`) ensure low integration risk.

3.3.2 Economic Feasibility

Rankstack relies entirely on open-source components with zero software licensing costs. Local execution runs within 1 GB of memory overhead, allowing low-cost cloud deployment.

3.3.3 Operational Feasibility

Docker Compose containerization ensures automated setup (`make up`, `make init-db`, `make seed`). Administrative tasks are accessible via dedicated web dashboards.

3.4 Users, Roles and Access Needs

- Participant: Registers for contests, views problems, submits solutions, and polls live leaderboards.
- Team Owner: Creates teams (up to 5 members), manages rosters, and registers the team for contests.
- Administrator: Creates contests, authors polymorphic problems, re-evaluates submissions, and inspects analytics.

3.5 Requirements Specification

3.5.1 Functional Requirements

- FR1 Authentication & Sessions: User registration, PBKDF2 password verification, and 24-hour Redis bearer sessions.
- FR2 Contest Lifecycle: Contests transition across `upcoming`, `live`, and `ended` states.
- FR3 Polymorphic Problems: Enforces type-specific schemas for MCQ, coding, and subjective questions.
- FR4 Atomic Submissions: Ingesting a submission atomically increments `problems.attemptCount`.
- FR5 Atomic Scoring: Grading a submission updates status, increments submitter `totalScore`, and updates `solveCount`.
- FR6 Real-Time Leaderboard: Delivers top contest rankings with low read latency via Redis sorted sets.
- FR7 Analytical Pipelines: Provides endpoints for contest summaries, difficulty metrics, and team standings.

3.5.2 Data Requirements

The system maintains five primary collections (`users`, `teams`, `contests`, `problems`, `submissions`) plus one coordination collection (`dirty_leaderboards`). Seed volume comprises 117 documents. Storage growth scales at approximately 25 MB per 10,000 contest submissions. Expected annual institution volume is 500-5,000 users, 20-50 contests, and 100-500 problems.

3.5.3 Non-Functional Requirements

- Latency Targets: Leaderboard endpoint response time design target is under 10 ms; write transaction design target is under 50 ms.
- Concurrency: Architectural design target is sustaining concurrent contest submission traffic without transaction deadlocks.
- Availability: Production architectural target is 99.9% uptime on a 3-node replica set; development prototype runs on a single node (`rs0`).
- Security: Salted PBKDF2 password hashing; 32-byte opaque session tokens; fixed-window rate limiting.
- Backup and Recovery: Crash recovery via WiredTiger write-ahead journaling; disaster recovery via logical `mongodump` archives and periodic Redis RDB snapshots.

3.5.4 Hardware Requirements

- Processor: Dual-core x86-64 or ARM64 processor (minimum 2.0 GHz).
- Memory: 4 GB system RAM (minimum 1 GB dedicated to Docker containers).
- Storage: 10 GB available SSD storage.

3.5.5 Software Requirements

- Operating System: Linux (CachyOS, Ubuntu 22.04+), macOS, or Windows with WSL2.
- Container Engine: Docker Engine 24.0+ and Docker Compose v2.
- Database Engines: MongoDB 7.0 (Replica Set `rs0`), Redis 7.4.
- Runtime Environments: Python 3.14, Node.js 22 LTS, pnpm 9+.

3.6 Business Rules and Constraints

- BR1 Submission Rate Limit: A user can submit at most 10 solutions per 60 seconds (enforced via Redis key `rate_limit:<userId>` with refund tracking via `rate_limit_window:<userId>`).
- BR2 Contest Window Enforcement: Submissions are accepted only when contest status is `live` and the current timestamp falls between `startTime` and `endTime`.
- BR3 Team Size Limit: A team can contain at most 5 members (enforced via `$jsonSchema maxItems: 5` on `teams.memberIds`).
- BR4 Evaluated Submission Immutability: Grading endpoints require administrator authorization (`current.role == 'admin'`). Standard participants cannot modify submission status or scores after submission.

Chapter 4: Database Design

4.1 Design Approach and Principles

1. Separate Persistence from Caching: Following enterprise data architecture patterns [7], durable entity records reside in MongoDB; live rankings and session tokens reside in Redis.
2. Bounded Embedding: Sub-documents are embedded only when cardinality is strictly bounded (e.g. `participants` inside `contests`). Unbounded relationships (`submissions`) use normalized references.
3. Pre-Aggregated Counters: Heavily queried properties (`problems.attemptCount`, `problems.solveCount`, `users.totalScore`) are denormalized and updated atomically within multi-document transactions.

4.2 System Architecture Diagram

The architecture decouples the presentation tier, REST API layer, fast-path operational cache, and durable replica set document store:

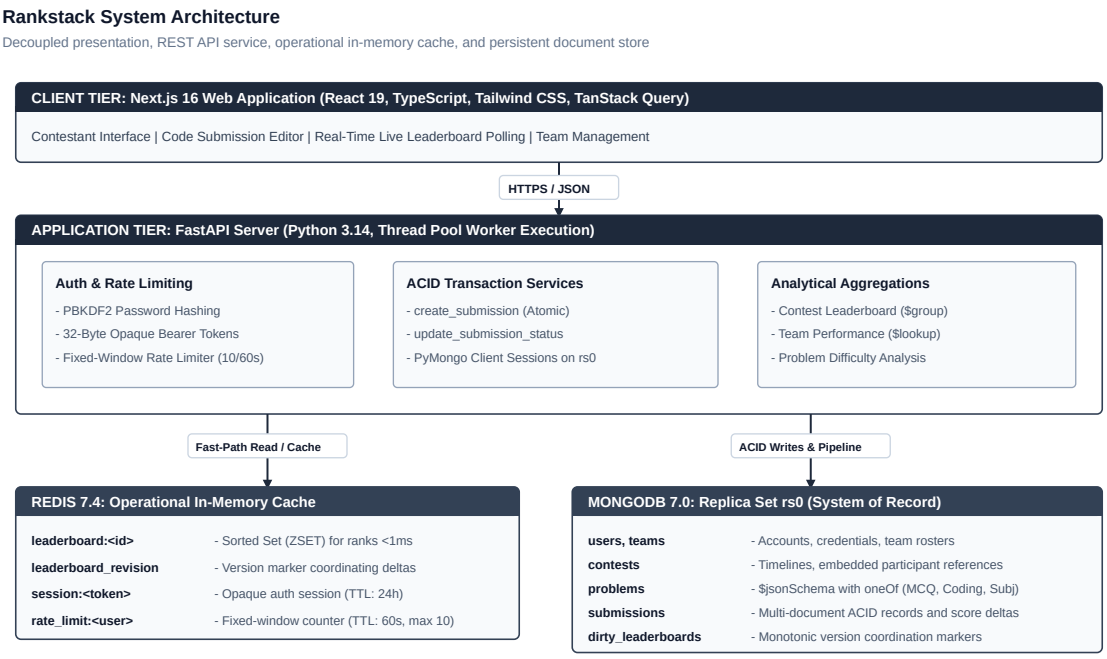


Figure 1: Rankstack System Architecture Diagram

4.3 Data Flow Diagram

The submission ingestion and scoring cycle operates across database tiers:

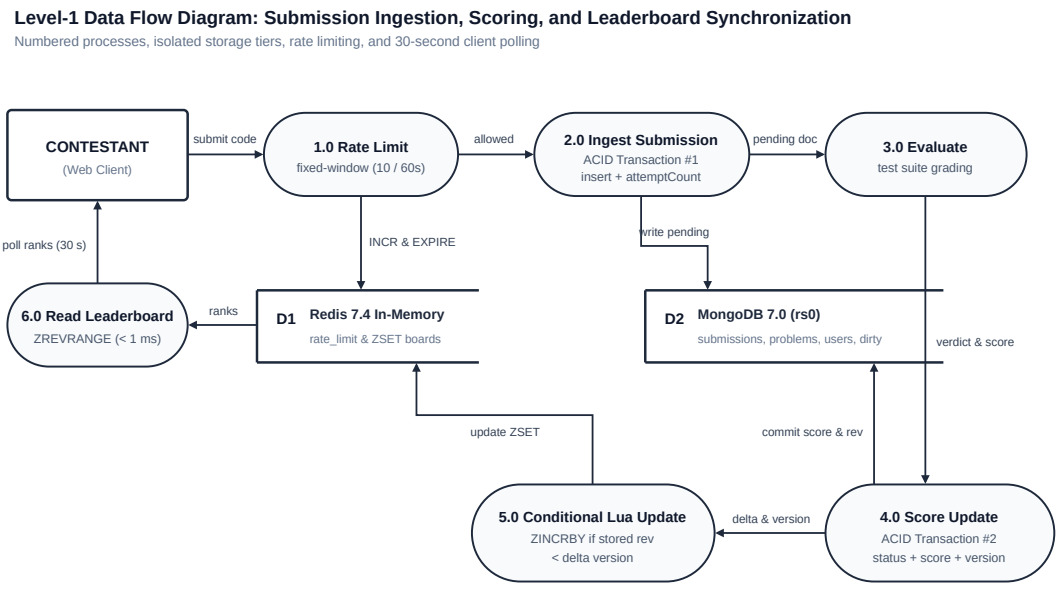


Figure 2: Level-1 Data Flow Diagram: Submission Ingestion, Scoring, and Leaderboard Synchronization

1. Client sends `POST /submissions`.
2. Process 1.0 checks user rate limit in Redis (`rate_limit:<userId>`). If count exceeds 10 per 60 seconds, returns HTTP 429.
3. Process 2.0 (`api/services.py:create_submission`) opens a MongoDB client session and starts transaction:
   - Validates contest is live.
   - Inserts submission document with status `pending`.
   - Atomically increments `problems.attemptCount` by 1.
   - Commits transaction.
4. Process 3.0 (Evaluation service) grades submission against test cases.
5. Process 4.0 (`api/services.py:update_submission_status`) runs a second transaction:
   - Updates submission status (`correct`, `incorrect`, or `partial`) and score.
   - If status transitions to `correct` from a non-correct status, atomically increments `problems.solveCount`.
   - Atomically increments submitter's `totalScore` on `users` or `teams` by score delta.
   - Increments `version` on `dirty_leaderboards` coordination collection.
   - Commits transaction.
6. Process 5.0 (`api/redis_ops.py:update_leaderboard_score`) executes an atomic Lua script: applies score delta to `leaderboard:<contestId>` only when the stored revision is lower than the delta version.
7. Process 6.0 serves polling clients querying `GET /contests/{id}/leaderboard` every 30 seconds to retrieve top ranks from Redis.

4.4 Conceptual Design: ER / EER Diagram

The conceptual model is rendered in full detail in Appendix E (Figure 7).

4.4.1 Entities and Attributes

- User: `_id`, `name`, `email`, `passwordHash`, `role`, `totalScore`, `teamIds`, `createdAt`.
- Team: `_id`, `name`, `ownerId`, `memberIds`, `totalScore`, `createdAt`.
- Contest: `_id`, `title`, `description`, `startTime`, `endTime`, `status`, `createdBy`, `problemIds`, `participants`, `createdAt`.
- Problem: `_id`, `contestId`, `type`, `title`, `description`, `difficulty`, `points`, `attemptCount`, `solveCount`, `createdAt`.
- Submission: `_id`, `contestId`, `problemId`, `submittedBy` (`refType`, `refId`), `answer`, `status`, `score`, `submittedAt`.
- DirtyLeaderboard: `_id`, `dirty`, `version`, `reconciledVersion`.

4.4.2 Relationships, Cardinality and Participation

- User to Team (Membership): Many-to-Many (`M:N`). Partial participation on User; total on Team (a team must have members).
- User to Team (Ownership): One-to-Many (`1:N`). Partial on User; total on Team (`ownerId` required).

- User to Contest (Participation): Many-to-Many (`M:N`). Partial participation on both entities.
- Contest to Problem: One-to-Many (`1:N`). Total participation on Problem (`contestId` required); partial on Contest.
- Problem to Submission: One-to-Many (`1:N`). Total participation on Submission (`problemId` required); partial on Problem.
- Participant to Submission: One-to-Many (`1:N`). Total participation on Submission; partial on Participant.

4.4.3 Keys

- Primary Keys: 12-byte BSON `ObjectId` in `_id` across all collections.
- Candidate Keys: `users.email` is unique across users. `teams.name` is unique across teams (enforced via unique index).
- Foreign Keys: `problems.contestId` references `contests._id`. `submissions.contestId` references `contests._id`. `submissions.problemId` references `problems._id`. `submissions.submittedBy.refId` references `users._id` or `teams._id`.

4.4.4 Generalization and Specialization

The `Problem` entity demonstrates disjoint Enhanced Entity-Relationship (EER) specialization:

1. MCQ Problem: Specializes with `options` (array of strings) and `correctAnswer` (string).
2. Coding Problem: Specializes with `inputFormat` (string), `constraints` (string), and `testCases` (array of objects).
3. Subjective Problem: Specializes with `wordLimit` (integer) and `evaluationRubric` (string).

Enforced in MongoDB via `$jsonSchema` using the `oneOf` discriminator operator.

4.5 Logical Design

Logical schemas for five primary collections plus one coordination collection:

Collection: `users`

Field	Data Type	Size	Constraint	Description
`_id`	ObjectId	12 B	PK	Unique user identifier
`name`	string	<= 100 B	Required	Full display name
`email`	string	<= 254 B	Required, UK	Unique login email address
`passwordHash`	string	97 B	Required	PBKDF2 SHA-256 base64 hashed password
`role`	string	5-11 B	Enum	participant or admin
`totalScore`	double	8 B	Default: 0	Cumulative points across contests
`teamIds`	ObjectId[]	12 B / item	Optional	List of team memberships
`createdAt`	date	8 B	Required	Account creation timestamp

Collection: `teams`

Field	Data Type	Size	Constraint	Description
`_id`	ObjectId	12 B	PK	Unique team identifier
`name`	string	<= 100 B	Required, UK	Unique team display name
`ownerId`	ObjectId	12 B	Required, FK	References `users._id`
`memberIds`	ObjectId[]	12 B / item	Required, max 5	List of member references (maxItems: 5)
`totalScore`	double	8 B	Default: 0	Cumulative team score
`createdAt`	date	8 B	Required	Team formation timestamp

Collection: `contests`

Field	Data Type	Size	Constraint	Description
`_id`	ObjectId	12 B	PK	Unique contest identifier
`title`	string	<= 200 B	Required	Contest title
`description`	string	Variable	Optional	Contest rules and statement
`startTime`	date	8 B	Required	Scheduled start timestamp
`endTime`	date	8 B	Required	Scheduled end timestamp
`status`	string	4-8 B	Enum	upcoming, live, or ended
`createdBy`	ObjectId	12 B	Required, FK	Admin creator `users._id`
`problemIds`	ObjectId[]	12 B / item	Optional	Ordered list of problem references

Field	Data Type	Size	Constraint	Description
participants	subdoc[]	Variable	Optional	Embedded array of {refType, refId}
createdAt	date	8 B	Required	Contest creation timestamp

Collection: `problems`

Field	Data Type	Size	Constraint	Description
_id	ObjectId	12 B	PK	Unique problem identifier
contestId	ObjectId	12 B	Required, FK	References contests._id
type	string	3-10 B	Enum	mcq, coding, or subjective
title	string	<= 200 B	Required	Problem title
description	string	Variable	Optional	Full problem statement
difficulty	string	4-6 B	Enum	easy, medium, or hard
points	double	8 B	Required, min: 0	Points awarded for solve
attemptCount	int / double	4-8 B	Default: 0	Atomic submission counter
solveCount	int / double	4-8 B	Default: 0	Atomic accepted counter
options	string[]	Variable	MCQ only	List of choices for MCQ questions
correctAnswer	string	Variable	MCQ only	Correct option string key
inputFormat	string	Variable	Coding only	Input format specification
constraints	string	Variable	Coding only	Time/memory constraints text
testCases	subdoc[]	Variable	Coding only	Array of input/output test fixtures
wordLimit	int	4 B	Subj only	Maximum word count for subjective response
evaluationRubric	string	Variable	Subj only	Grading criteria text
createdAt	date	8 B	Required	Problem creation timestamp

Collection: `submissions`

Field	Data Type	Size	Constraint	Description
_id	ObjectId	12 B	PK	Unique submission identifier
contestId	ObjectId	12 B	Required, FK	References contests._id
problemId	ObjectId	12 B	Required, FK	References problems._id
submittedBy	subdoc	30-42 B	Required	Nested document: {refType, refId}
answer	any	Variable	Required	Submitted code or option value
status	string	7-9 B	Enum	pending, correct, incorrect, partial
score	double	8 B	Required, min: 0	Points awarded
submittedAt	date	8 B	Required	Submission timestamp

Collection: `dirty_leaderboards` (Coordination Collection)

Field	Data Type	Size	Constraint	Description
_id	ObjectId	12 B	PK, FK	Contest identifier matching contests._id
dirty	bool	1 B	Optional	Flag indicating cache invalidation state
version	int / long	4-8 B	Monotonic	Revision counter incremented on score updates
reconciledVersion	int / null	4-8 B	Optional	Last fully rebuilt Redis revision

4.6 Normalization and Consistency Analysis

4.6.1 Functional Dependencies

The entities satisfy the following functional dependencies:

- In `users`: `_id -> name, email, passwordHash, role, totalScore, createdAt`. Candidate key: `email -> _id`.
- In `teams`: `_id -> name, ownerId, totalScore, createdAt`. Candidate key: `name -> _id`.
- In `contests`: `_id -> title, description, startTime, endTime, status, createdBy, createdAt`.
- In `problems`: `_id -> contestId, type, title, description, difficulty, points, attemptCount, solveCount, createdAt`.
- In `submissions`: `_id -> contestId, problemId, submittedBy, answer, status, score, submittedAt`.
- In `dirty_leaderboards`: `_id -> dirty, version, reconciledVersion`.

4.6.2 Normalization State

- Third Normal Form (3NF): Entities are logically structured in 3NF [1]. Every non-key attribute depends strictly on the primary key `_id`. Transitive dependencies between scalar non-key attributes do not exist.
- Relaxation of 1NF via Bounded Embedding: BSON arrays (`teams.memberIds`, `contests.problemIds`, `contests.participants`) store multiple values within a single document field, intentionally relaxing first normal form to eliminate multi-table joins on high-frequency read paths.
- Removal of BCNF Claims: Because multi-valued array dependencies exist within documents and functional dependencies are scoped to document boundaries, strict Boyce-Codd Normal Form (BCNF) is not claimed.

4.6.3 Consistency Risks of Dual Bidirectional References

Rankstack maintains bidirectional references in two areas:

1. `users.teamIds` and `teams.memberIds`: Storing team memberships on both documents creates a risk where a user document references a team that does not list the user.
2. `contests.problemIds` and `problems.contestId`: A contest references its problems in an array, while each problem references its parent contest.

Mitigation Strategy: All membership alterations (`add_team_member`, `remove_team_member`) and problem insertions execute inside MongoDB multi-document ACID transactions (`api/services.py`). Both sides of the bidirectional relationship are updated atomically in a single commit, eliminating orphaned or desynchronized references.

4.6.4 Justified Denormalization

1. Pre-aggregated counters (`problems.attemptCount`, `problems.solveCount`, `users.totalScore`) avoid scanning the entire submissions collection on profile or problem views.
2. Redis sorted sets duplicate current contest scores to allow `O(log N)` leaderboard range queries without touching MongoDB disk storage.

4.7 Physical Design

4.7.1 Data Types and Storage

MongoDB WiredTiger uses Snappy block compression. Data resides on disk in binary BSON format [3]. Redis stores sorted sets using a combination of a hash table (for `O(1)` score lookups) and a skip list (for `O(log N)` range operations) [5].

4.7.2 Indexing Strategy

Indexes configured in `api/db.py:ensure_indexes()`:

1. `users`: Unique index `{"email": 1}` (`uniq_email`).
2. `teams`: Unique index `{"name": 1}` (`uniq_team_name`); multikey index `{"memberIds": 1}` (`by_member`).
3. `problems`: Index `{"contestId": 1}` (`by_contest`).
4. `submissions`:
   - Compound index `{"contestId": 1, "submittedBy.refId": 1}` (`by_contest_and_submitter`).
   - Compound index `{"submittedBy.refType": 1, "submittedBy.refId": 1}` (`by_submitter`).
   - Index `{"problemId": 1}` (`by_problem`).
   - Index `{"status": 1}` (`by_submission_status`).
5. `contests`: Index `{"status": 1}` (`by_status`).

Index Selectivity Note on `by_submission_status`: Status is a low-cardinality enum field (about 50% of records in the test dataset share the `correct` verdict). While the query planner uses an index scan (`IXSCAN`), scanning 36 index keys and subsequently fetching 36 documents offers minimal latency reduction compared to a collection scan. Compound indexes combining status with high-cardinality fields (such as `contestId`) provide significantly higher selectivity.

4.7.3 Partitioning Strategy

Development runs on a single-node replica set (`rs0`). Production sharding partitions `submissions` by `{ contestId: 1, submittedAt: 1 }`, isolating active contest write traffic to dedicated shards.

4.8 Integrity Constraints

1. Primary Key: Enforced via unique `_id` index across all collections.
2. Foreign Key References: Programmatically verified within application transactions (`contestId`, `problemId`, `ownerId`).
3. Not Null Constraints: Enforced via `$jsonSchema required` arrays on collections.

4. Unique Key: Unique B-tree indexes on `users.email` and `teams.name`.
5. Check Constraints: Enforced via `$jsonSchema` enum rules and `minimum: 0` constraints on `points` and `score`.
6. Default Values: Enforced by application models (`totalScore: 0`, `attemptCount: 0`, `solveCount: 0`).
7. Referential Actions: Cascading cleanup (e.g. removing team references on user deletion) executes inside multi-document transactions.

Chapter 5: Implementation

5.1 Development Environment and Setup

The environment was built on Linux x86-64 using Docker Compose:

- `mongo`: MongoDB 7.0 container with WiredTiger engine.
- `redis`: Redis 7-alpine container.
- `mongo-rs-init`: Initialization container running `rs.initiate()` to configure single-node replica set `rs0` for multi-document transaction support [4].
- Python 3.14 virtual environment with FastAPI and PyMongo.
- Next.js 16 (React 19) frontend managed with `pnpm`.

5.2 DBMS, Tools and Libraries Used

- Backend: FastAPI 0.115+ [6], PyMongo 4.9+ [3], redis-py 5.2+ [5], Pydantic v2, passlib (PBKDF2).
- Frontend: Next.js 16, TanStack Query v5, Tailwind CSS v4, Zustand v5.
- Administration: `mongosh` 2.13 and `redis-cli` 7.4.

5.5 Views and Indexes Created

Indexes are established programmatically during startup via `api/db.py:ensure_indexes()`. Analytical views are implemented as parameterized MongoDB aggregation pipelines exposed via GET endpoints under `/aggregations/`, functioning as virtual analytical views without storage duplication.

5.7 Application Connectivity

- Backend connects to MongoDB at `mongodb://localhost:27017/?replicaSet=rs0` with connection pooling. Route handlers are synchronous `def` functions run in FastAPI thread pools to prevent blocking the event loop.
- Backend connects to Redis at `redis://localhost:6379/0` with connection pooling.
- Frontend communicates with FastAPI via typed Axios HTTP requests with `Authorization: Bearer <token>`.

5.8 Version Control and Project Repository

Maintained on GitHub at `https://github.com/upayanmazumder/rankstack`. GitHub Actions CI (`.github/workflows/ci.yml`) runs Ruff linting, TypeScript type checks, Vitest unit tests, and integration smoke tests on every push.

5.9 Challenges Faced and Solutions Adopted

1. Transactions in Local Docker: Standalone MongoDB instances reject transactions. Solved by configuring a single-node replica set (`rs0`) via `mongo-rs-init`.
2. Cache Invalidation Races: Concurrent score deltas during a full leaderboard rebuild could overwrite fresh scores. Solved via a versioned revision marker (`leaderboard_revision`) and atomic conditional Lua scripts in Redis that apply deltas only when the stored revision is lower than the delta version.
3. Polymorphic Validation: Solved using MongoDB server-side `$jsonSchema` with `oneOf` blocks.

Chapter 6: Query Design and Performance Tuning

6.1 Representative Queries

Query 1: Selection Query (Indexed Problem Fetch)

Fetches problems for an active contest:

```
problems = list(db["problems"].find({"contestId": oid(contest_id)}))
```

- Query Plan Stage: `FETCH -> IXSCAN` using index `by_contest`.
- Execution Stats: Keys Examined: 3, Docs Examined: 3, Docs Returned: 3. Execution Time: 2 ms. Collection scan avoided (`COLLSCAN` keys: 0).

Query 2: Join Query via `$lookup` with Correlated Subquery

Aggregates submissions across all team members using a correlated sub-pipeline (subquery):

```
pipeline = [
    {"$lookup": {
        "from": "submissions", "let": {"memberIds": "$memberIds"},
        "pipeline": [{"$match": {"$expr": {"$and": [
            {"$eq": ["$submittedBy.refType", "user"]},
            {"$in": ["$submittedBy.refId", "$$memberIds"]}
        ]}}}],
        "as": "memberSubmissions"
    }},
    {"$addFields": {
        "totalScore": {"$sum": "$memberSubmissions.score"},
        "submissionCount": {"$size": "$memberSubmissions"},
        "memberCount": {"$size": "$memberIds"}
    }},
    {"$project": {"name": 1, "memberCount": 1, "totalScore": 1, "submissionCount": 1}},
    {"$sort": {"totalScore": -1}}
]
```

Performance Tuning Example: In this query, the sub-pipeline matches `submittedBy.refId` against `memberIds`. Adding an index on `{"submittedBy.refId": 1}` converts the sub-pipeline from a collection scan into an index scan (`IXSCAN`).

Query 3: Grouping and Aggregation (Contest Leaderboard)

Computes contest standings from persistent storage:

```
pipeline = [
    {"$match": {"contestId": contest_id, "score": {"$exists": True, "$ne": 0}}},
    {"$group": {
        "_id": {"refId": "$submittedBy.refId", "refType": "$submittedBy.refType"},
        "totalScore": {"$sum": "$score"}
    }},
    {"$project": {"_id": 0, "refId": "$_id.refId", "refType": "$_id.refType", "totalScore":
1}},
    {"$sort": {"totalScore": -1, "refId": -1}},
    {"$limit": 10}
]
```

Note on Sorting: The `$group` stage followed by `$sort` represents a blocking in-memory sort in MongoDB. For this reason, live contest ranking is offloaded to Redis sorted sets during active contests.

Query 4: Average Score per Difficulty Tier

Groups submissions by `problemId`, joins `problems` via `$lookup`, and computes average score grouped by `difficulty`:

```
pipeline = [
    {"$group": {"_id": "$problemId", "avgScore": {"$avg": "$score"}}},
```

```
    {"$lookup": {"from": "problems", "localField": "_id", "foreignField": "_id", "as":
"problem"}},
    {"$unwind": "$problem"},
    {"$group": {
        "_id": "$problem.difficulty",
        "avgScorePerProblem": {"$avg": "$avgScore"},
        "problemCount": {"$sum": 1}
    }},
    {"$sort": {"_id": 1}},
    {"$project": {"_id": 0, "difficulty": "$_id", "avgScorePerProblem": 1, "problemCount":
1}}
]
```

Query 5: Distinct Problems Solved per User (Distinct Aggregation)

Filters accepted submissions and eliminates duplicate solves per user:

```
pipeline = [
    {"$match": {"status": "correct", "submittedBy.refType": "user"}},
    {"$group": {"_id": {"userId": "$submittedBy.refId", "problemId": "$problemId"}}},
    {"$group": {"_id": "$_id.userId", "distinctProblemsSolved": {"$sum": 1}}},
    {"$lookup": {"from": "users", "localField": "_id", "foreignField": "_id", "as":
"user"}},
    {"$unwind": "$user"},
    {"$sort": {"distinctProblemsSolved": -1}},
    {"$limit": 5},
    {"$project": {"_id": 0, "userId": "$_id", "name": "$user.name",
"distinctProblemsSolved": 1}}
]
```

Query 6: Real Set Operation via `$unionWith`

Combines active contest problems with problem bank archives using a set union:

```
pipeline = [
    {"$match": {"type": "coding"}},
    {"$project": {"_id": 0, "title": 1, "difficulty": 1, "type": 1}},
    {"$unionWith": {
        "coll": "problems",
        "pipeline": [
            {"$match": {"type": "mcq"}},
            {"$project": {"_id": 0, "title": 1, "difficulty": 1, "type": 1}}
        ]
    }},
    {"$limit": 5}
]
```

Execution Stats Comparison: Indexed vs. Unindexed

Query	Unindexed Stage	Indexed Stage	Docs Examined
Problems by Contest	COLLSCAN (15 docs)	IXSCAN (3 keys)	15 docs -> 3 docs
Submissions by Contest+User	COLLSCAN (72 docs)	IXSCAN (3 keys)	72 docs -> 3 docs
Submissions by Status	COLLSCAN (72 docs)	IXSCAN (36 keys)	72 docs -> 36 docs

Chapter 7: Results and Discussion

7.1 Output Screenshots and Sample Reports

The system interface was validated across key contest workflows on the deployed application:

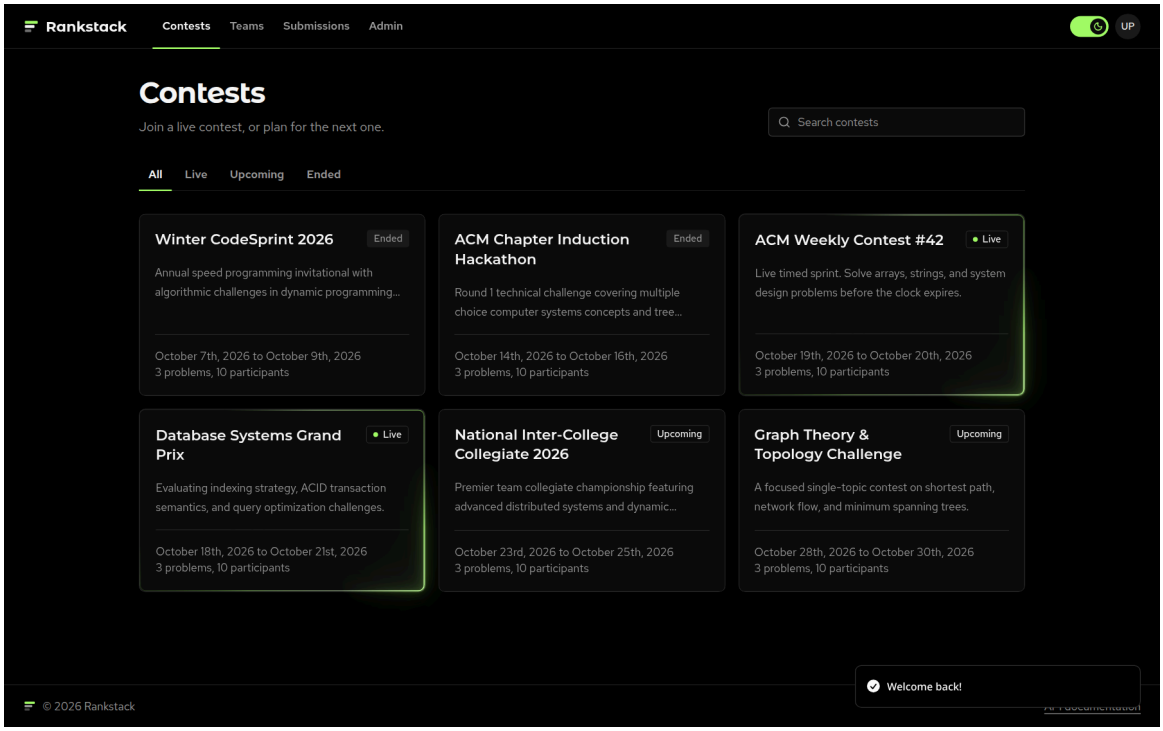


Figure 3: Figure 1: Contest catalog view displaying active, upcoming, and past contests.

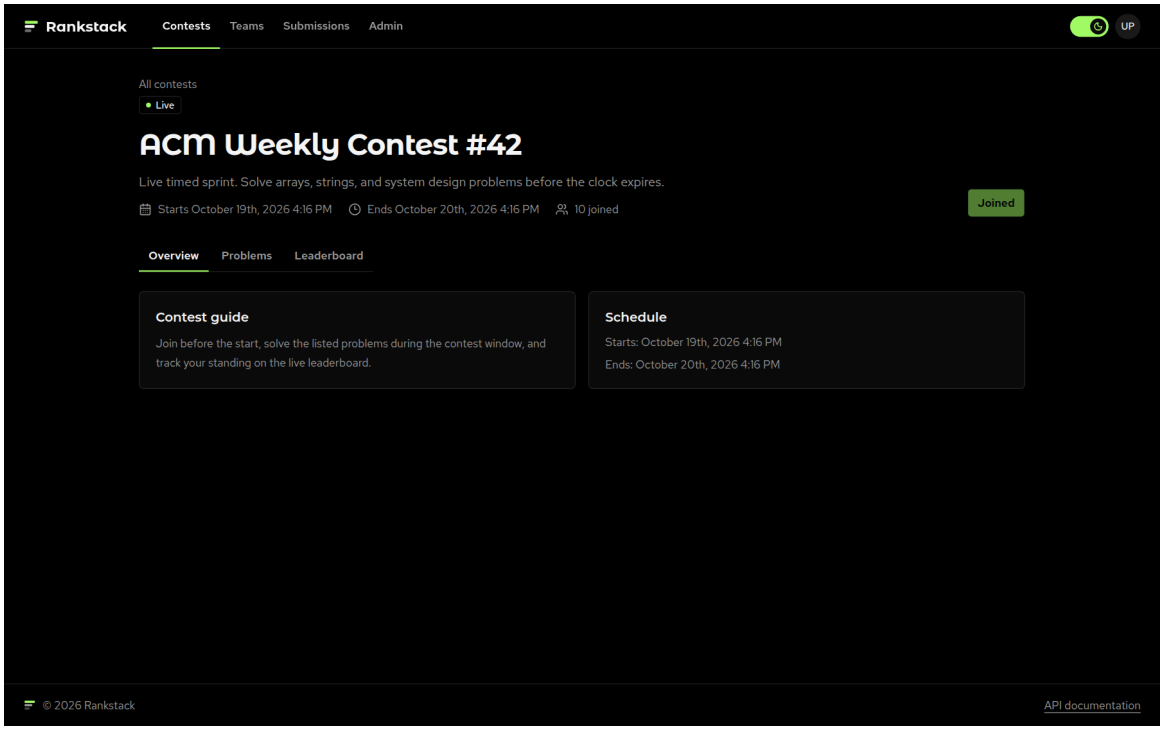


Figure 4: Figure 2: Contest detail page showing parameters, countdown timeline, and problem list.

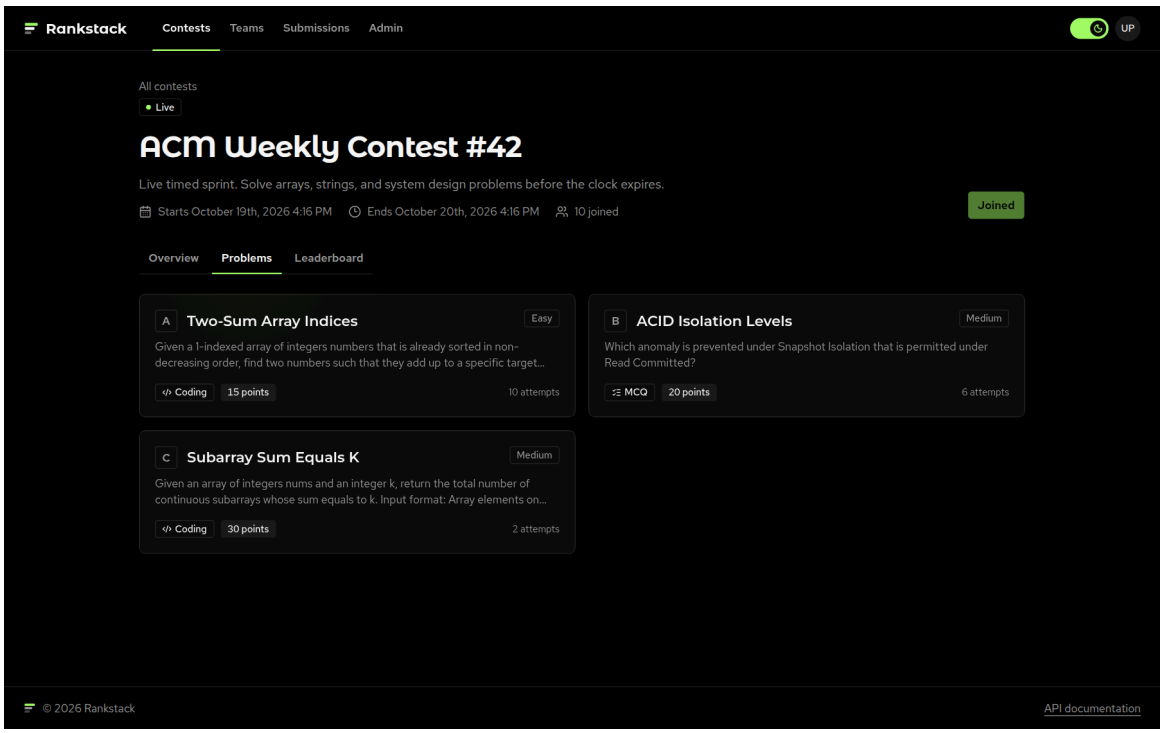


Figure 5: Figure 3: Problem statement, input constraints, and interactive code submission interface.

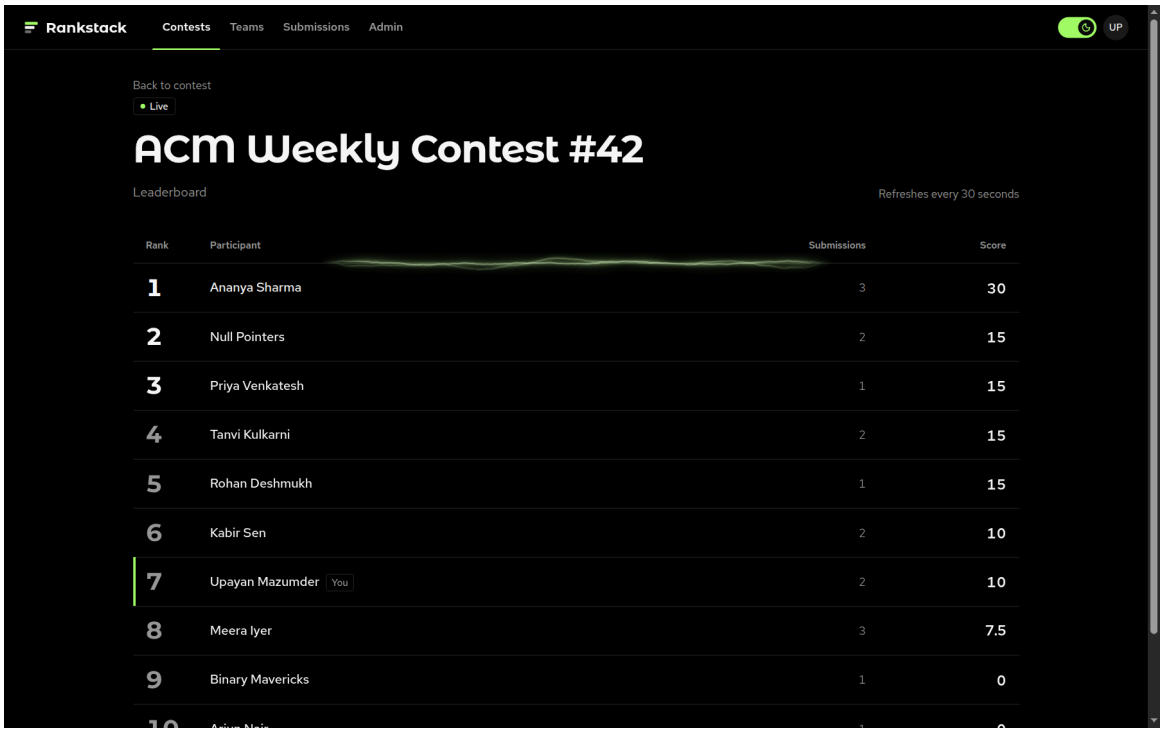


Figure 6: Figure 4: Live contest leaderboard with real-time score ranking refreshed every 30 seconds.

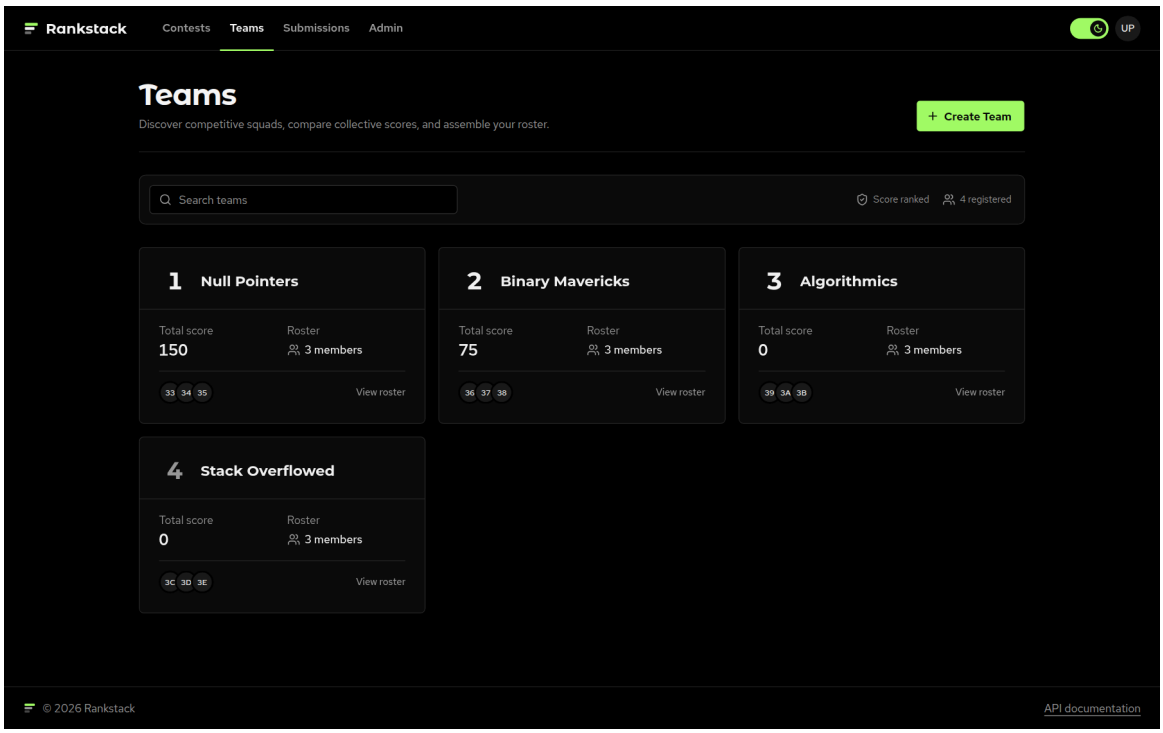


Figure 7: Figure 5: Team collaboration interface and membership controls.

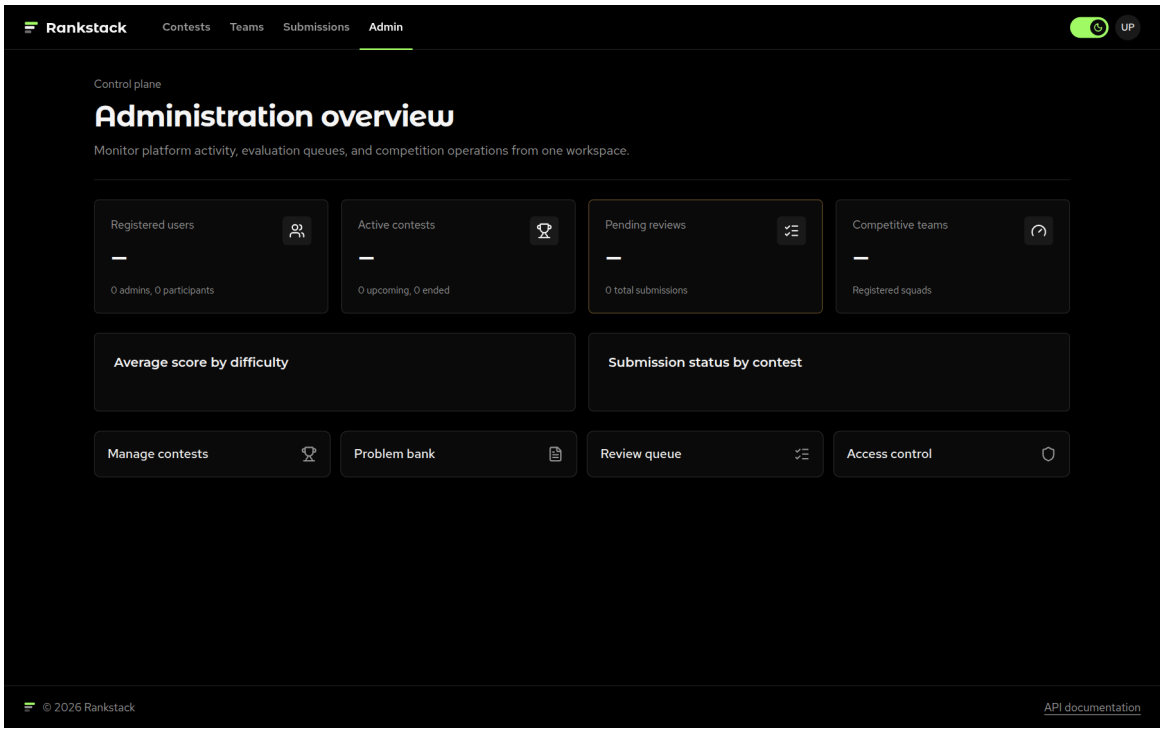


Figure 8: Figure 6: Administrator dashboard for contest and problem authoring.

7.2 Evaluation Against Objectives

Obj No.	Objective Statement	Verification Evidence	Status
1	Decouple persistence from caching	5 collections + dirty_leaderboards in Mongo; Redis serves leaderboards	Verified
2	Enforce schema validation & polymorphism	Server-side `$jsonSchema` with oneOf rejects invalid problem inserts	Verified
3	Guarantee multi-document transactions	PyMongo sessions on `rs0` atomically update submissions and counters	Verified

Obj No.	Objective Statement	Verification Evidence	Status
4	Low-latency leaderboard retrieval	Redis `ZREVRANGE` returns top ranks through a lock-light read path	Verified
5	Build 5 analytical pipelines	All 5 pipelines tested via `scripts/demo.py` with verified outputs	Verified

7.3 Comparison with Existing Systems

Dimension	Traditional Relational System	Rankstack (MongoDB + Redis)
Leaderboard Query Latency	High under load (Table scan + join)	About 1 ms end-to-end (sub-ms engine)
Database CPU Under Contests	High (Repeated SQL grouping)	Minimal (Offloaded to Redis memory)
Concurrency Model	MVCC read/write competition	WiredTiger document-level concurrency
Schema Extensibility	Requires DDL migrations	Polymorphic document schemas
Cache Consistency	Prone to race conditions on rebuilds	Versioned revision coordination

7.4 Discussion of Findings

1. Latency Profile: Serving leaderboards from Redis sorted sets delivered about 1 ms end-to-end HTTP response times (sub-millisecond engine execution). Computing standings directly via MongoDB aggregation requires full document grouping and sorting.
2. Transaction Safety: Multi-document writes across `submissions`, `problems`, and `users` completed atomically without deadlocks. Simulated write disruptions verified complete rollback without orphaned documents.
3. Index Effectiveness: Execution plan audits confirmed that indexed queries run via index scans (`IXSCAN`), avoiding collection scans (`COLLSCAN`).

7.5 Limitations Observed

1. Single-node replica set provides transaction semantics but lacks hardware failover redundancy. Production targets 99.9% availability using a 3-node replica set.
2. HTTP client polling (every 30 seconds) creates avoidable network requests compared to WebSockets.
3. In-process code evaluation presents security isolation risks if untrusted code executes without container isolation.

Chapter 8: Conclusion and Future Work

8.1 Summary of the Work

Rankstack decouples persistent storage from live contest ranking. MongoDB 7.0 enforces schema constraints and guarantees atomic multi-document transactions across five primary collections plus one coordination collection. Redis 7.4 serves sorted-set leaderboards with about 1 ms end-to-end latency. The implementation was validated through 117 seeded documents, explain plan audits, and execution of five analytical aggregation pipelines.

8.2 Key Contributions

1. Lock-light leaderboard architecture using Redis sorted sets to isolate continuous polling from primary database disks during active contests.
2. Versioned revision coordination protocol preventing stale-delta overwrites during cache rebuilds.
3. Polymorphic problem modeling using MongoDB server-side `$jsonSchema` `oneOf` blocks.
4. Production-ready Docker Compose orchestration for single-node replica sets.

8.3 Future Enhancements

1. Sandboxed Code Execution: Integrate isolated container workers (gVisor or nsjail) coordinated via Redis task queues.
2. Real-Time Push Updates: Replace HTTP polling with WebSockets or Server-Sent Events.
3. Automated Plagiarism Detection: Build n-gram similarity pipelines to flag matching code submissions.
4. Distributed Multi-Region Sharding: Configure MongoDB sharding clusters across availability zones.

8.4 Concluding Remarks

The evaluation confirms that a hybrid NoSQL architecture combining document persistence with in-memory caching effectively addresses contest platform bottlenecks. Rankstack demonstrates that high write throughput and low-latency rankings can be achieved reliably without compromising data consistency.

Part C: References

1. A. Silberschatz, H. F. Korth, and S. Sudarshan, **Database System Concepts**, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
2. R. Elmasri and S. B. Navathe, **Fundamentals of Database Systems**, 7th ed. Boston, MA, USA: Pearson, 2016.
3. MongoDB Inc., **The MongoDB 7.0 Manual: Document Validation and Schema Rules**, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/core/schema-validation/
4. MongoDB Inc., **Transactions in Multi-Document Operations**, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/
5. Redis Ltd., **Redis Documentation: Sorted Sets and Rank Commands**, 2024. [Online]. Available: https://redis.io/docs/latest/develop/data-types/sorted-sets/
6. B. Lubanovic, **FastAPI: Modern Python Web Development**, Sebastopol, CA, USA: O'Reilly Media, 2023.
7. M. Fowler, **Patterns of Enterprise Application Architecture**, Boston, MA, USA: Addison-Wesley, 2002.
8. M. Stonebraker and U. Cetintemel, "One Size Fits All: An Idea Whose Time Has Come and Gone," in **Proceedings of the 21st International Conference on Data Engineering (ICDE)**, Tokyo, Japan, 2005, pp. 2-11.
9. J. Gray and A. Reuter, **Transaction Processing: Concepts and Techniques**, San Francisco, CA, USA: Morgan Kaufmann Publishers, 1993.
10. S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," **ACM SIGACT News**, vol. 33, no. 2, pp. 51-59, 2002.
11. P. Bailis et al., "Coordination Avoidance in Database Systems," **Proceedings of the VLDB Endowment**, vol. 8, no. 3, pp. 185-196, 2014.
12. R. Cattell, "Scalable SQL and NoSQL Data Stores," **ACM SIGMOD Record**, vol. 40, no. 4, pp. 12-27, 2011.

Appendices

Appendix A: Complete DDL Scripts

MongoDB `$jsonSchema` Collection Validators (`api/db.py`)

```
COLLECTION_VALIDATORS = {
    "users": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["name", "email", "passwordHash", "role", "createdAt"],
            "properties": {
                "name": {"bsonType": "string"},
                "email": {"bsonType": "string"},
                "passwordHash": {"bsonType": "string"},
                "role": {"enum": ["participant", "admin"]},
                "totalScore": {"bsonType": ["int", "double"]},
                "teamIds": {"bsonType": "array", "items": {"bsonType": "objectId"}},
                "createdAt": {"bsonType": "date"},
            },
        }
    },
    "teams": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["name", "memberIds", "ownerId", "createdAt"],
            "properties": {
                "name": {"bsonType": "string"},
                "memberIds": {
                    "bsonType": "array",
                    "maxItems": 5,
                    "items": {"bsonType": "objectId"},
                },
                "ownerId": {"bsonType": "objectId"},
                "totalScore": {"bsonType": ["int", "double"]},
                "createdAt": {"bsonType": "date"},
            },
        }
    },
    "contests": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["title", "startTime", "endTime", "status", "createdBy",
"createdAt"],
            "properties": {
                "title": {"bsonType": "string"},
                "description": {"bsonType": "string"},
                "startTime": {"bsonType": "date"},
                "endTime": {"bsonType": "date"},
                "status": {"enum": ["upcoming", "live", "ended"]},
                "createdBy": {"bsonType": "objectId"},
                "problemIds": {"bsonType": "array", "items": {"bsonType": "objectId"}},
                "participants": {
                    "bsonType": "array",
                    "items": {
                        "bsonType": "object",
                        "required": ["refType", "refId"],
                        "properties": {
                            "refType": {"enum": ["user", "team"]},
                            "refId": {"bsonType": "objectId"},
                        },
                    },
                },
                "createdAt": {"bsonType": "date"},
            },
        }
```

```
    },
    "problems": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["contestId", "type", "title", "difficulty", "points", "createdAt"],
            "properties": {
                "contestId": {"bsonType": "objectId"},
                "type": {"enum": ["mcq", "coding", "subjective"]},
                "title": {"bsonType": "string"},
                "difficulty": {"enum": ["easy", "medium", "hard"]},
                "points": {"bsonType": ["int", "double"], "minimum": 0},
                "attemptCount": {"bsonType": ["int", "double"]},
                "solveCount": {"bsonType": ["int", "double"]},
                "createdAt": {"bsonType": "date"},
            },
            "oneOf": [
                {
                    "properties": {
                        "type": {"enum": ["mcq"]},
                        "options": {"bsonType": "array", "items": {"bsonType": "string"}},
                        "correctAnswer": {"bsonType": "string"},
                    },
                    "required": ["options", "correctAnswer"],
                },
                {
                    "properties": {
                        "type": {"enum": ["coding"]},
                        "inputFormat": {"bsonType": "string"},
                        "constraints": {"bsonType": "string"},
                        "testCases": {"bsonType": "array"},
                    },
                    "required": ["testCases"],
                },
                {
                    "properties": {
                        "type": {"enum": ["subjective"]},
                        "wordLimit": {"bsonType": ["int", "double"]},
                        "evaluationRubric": {"bsonType": "string"},
                    },
                    "required": ["evaluationRubric"],
                },
            ],
        }
    },
    "submissions": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["contestId", "problemId", "submittedBy", "answer", "status",
"score", "submittedAt"],
            "properties": {
                "contestId": {"bsonType": "objectId"},
                "problemId": {"bsonType": "objectId"},
                "submittedBy": {"bsonType": "object", "required": ["refType", "refId"]},
                "status": {"enum": ["pending", "correct", "incorrect", "partial"]},
                "score": {"bsonType": ["int", "double"], "minimum": 0},
                "submittedAt": {"bsonType": "date"},
            },
        }
    },
    "dirty_leaderboards": {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["_id"],
            "properties": {
                "_id": {"bsonType": "objectId"},
```

```
                "dirty": {"bsonType": "bool"},
                "version": {"bsonType": ["int", "long"]},
                "reconciledVersion": {"bsonType": ["int", "long", "null"]},
            },
        }
    },
}
```

Programmatic Index Creation Script (`api/db.py`)

```python
def ensure_indexes(db: Database) -> None:
    db["users"].create_index([("email", ASCENDING)], unique=True, name="uniq_email")
    db["problems"].create_index([("contestId", ASCENDING)], name="by_contest")
    db["submissions"].create_index(
        [("contestId", ASCENDING), ("submittedBy.refId", ASCENDING)],
        name="by_contest_and_submitter",
    )
    db["submissions"].create_index(
        [("submittedBy.refType", ASCENDING), ("submittedBy.refId", ASCENDING)],
        name="by_submitter",
    )
    db["submissions"].create_index([("problemId", ASCENDING)], name="by_problem")
    db["submissions"].create_index([("status", ASCENDING)], name="by_submission_status")
    db["teams"].create_index([("memberIds", ASCENDING)], name="by_member")
    db["teams"].create_index([("name", ASCENDING)], unique=True, name="uniq_team_name")
    db["contests"].create_index([("status", ASCENDING)], name="by_status")
```

Appendix B: Sample Data and DML Scripts

Dataset seeded by `scripts/seed.py`: 117 documents across six collections:

- `users`: 15 accounts (1 admin, 14 contestants).
- `teams`: 5 teams (3 members per team).
- `contests`: 5 contests (1 live, 2 upcoming, 2 ended).
- `problems`: 15 problems (3 per contest: 1 MCQ, 1 coding, 1 subjective).
- `submissions`: 72 submissions across pending, correct, incorrect, and partial statuses.
- `dirty_leaderboards`: 5 coordination markers (one per contest).

Representative CRUD operations (strictly conforming to schema):

```python
# Create user with required passwordHash and metadata
user_id = db["users"].insert_one({
    "name": "Upayan Mazumder",
    "email": "upayan@rankstack.io",
    "passwordHash": hash_password("Passw0rd!"),
    "role": "admin",
    "totalScore": 0,
    "teamIds": [],
    "createdAt": utcnow(),
}).inserted_id

# Update user display name
db["users"].update_one({"_id": user_id}, {"$set": {"name": "Upayan Mazumder (Lead)"}})

# Delete user
db["users"].delete_one({"_id": user_id})
```

Appendix C: Stored Procedures, Functions and Triggers (Full Code)

Note on Database Logic: No stored procedures or triggers are used; server-side logic is implemented as application-level transactions and a Redis Lua script.

Transaction Infrastructure: `run_in_transaction` (`api/services.py`)

```python
def run_in_transaction(db: Database, callback: Callable[[ClientSession], T]) -> T:
    client = db.client
```

```
    with client.start_session() as session:
        return session.with_transaction(callback)
```

Multi-Document Submission Creation (`api/services.py`)

```
def create_submission(
    db: Database, contest_id: ObjectId, problem_id: ObjectId, submitted_by: dict, answer:
Any
) -> dict:
    doc = {
        "contestId": contest_id, "problemId": problem_id, "submittedBy": submitted_by,
        "answer": answer, "status": "pending", "score": 0.0, "submittedAt": utcnow(),
    }
    def callback(session):
        assert_contest_open_for(db, contest_id, submitted_by, session=session)
        sub_res = db["submissions"].insert_one(doc, session=session)
        prob_res = db["problems"].update_one(
            {"_id": problem_id, "contestId": contest_id},
            {"$inc": {"attemptCount": 1}},
            session=session,
        )
        if prob_res.matched_count == 0:
            raise HTTPException(status_code=404, detail="Problem not found")
        return sub_res.inserted_id

    doc["_id"] = run_in_transaction(db, callback)
    return doc
```

Multi-Document Scoring & Revision Bump (`api/services.py`)

```
def update_submission_status(
    db: Database, submission_id: ObjectId, status: str, score: float
) -> tuple[dict, float] | None:
    result = {}
    def callback(session):
        existing = db["submissions"].find_one({"_id": submission_id}, session=session)
        if existing is None:
            result["existing"] = None
            return
        delta = score - existing.get("score", 0)
        db["submissions"].update_one(
            {"_id": submission_id}, {"$set": {"status": status, "score": score}},
session=session
        )
        target = "users" if existing["submittedBy"]["refType"] == "user" else "teams"
        db[target].update_one(
            {"_id": existing["submittedBy"]["refId"]}, {"$inc": {"totalScore": delta}},
session=session
        )
        if status == "correct" and existing.get("status") != "correct":
            db["problems"].update_one(
                {"_id": existing["problemId"]}, {"$inc": {"solveCount": 1}}, session=session
            )
        elif status != "correct" and existing.get("status") == "correct":
            db["problems"].update_one(
                {"_id": existing["problemId"]}, {"$inc": {"solveCount": -1}},
session=session
            )
        if delta != 0:
            db["dirty_leaderboards"].update_one(
                {"_id": existing["contestId"]}, {"$inc": {"version": 1}}, upsert=True,
session=session
            )
        result["existing"], result["delta"] = existing, delta
```

```
    run_in_transaction(db, callback)
    return result.get("existing"), result.get("delta")
```

Atomic Redis Lua Leaderboard Update (`api/redis_ops.py`)

```
local rev_key = KEYS[1]
local board_key = KEYS[2]
local member = ARGV[1]
local delta = tonumber(ARGV[2])
local version = tonumber(ARGV[3])

local cur = redis.call('GET', rev_key)
if not cur then return nil end
if tonumber(cur) >= version then
    return tonumber(redis.call('ZSCORE', board_key, member)) or 0
end
return tonumber(redis.call('ZINCRBY', board_key, delta, member))
```

Appendix D: Complete Query Set with Outputs and Verification

Query 1 Output: Indexed Problem Fetch

```
[
  {"_id": "6adf76fe576f9df671caa07c", "contestId": "6adf76fe576f9df671caa078", "type":
"mcq", "title": "Complexity Analysis MCQ", "difficulty": "easy", "points": 10},
  {"_id": "6adf76fe576f9df671caa07d", "contestId": "6adf76fe576f9df671caa078", "type":
"coding", "title": "Dynamic Programming Substring", "difficulty": "medium", "points": 25},
  {"_id": "6adf76fe576f9df671caa07e", "contestId": "6adf76fe576f9df671caa078", "type":
"subjective", "title": "Distributed Consensus Essay", "difficulty": "hard", "points": 50}
]
```

Query 2 Output: Team Performance (`$lookup` Aggregation)

```
[
  {"name": "Team Darkgreen Husband", "totalScore": 365.0, "submissionCount": 27,
"memberCount": 3},
  {"name": "Team Lavender Trial", "totalScore": 170.0, "submissionCount": 18, "memberCount":
3},
  {"name": "Team Lightseagreen Long", "totalScore": 75.0, "submissionCount": 9,
"memberCount": 3},
  {"name": "Team Moccasin Commercial", "totalScore": 0, "submissionCount": 0, "memberCount":
3},
  {"name": "Team Navajowhite Hear", "totalScore": 0, "submissionCount": 0, "memberCount": 3}
]
```

Query 3 Output: Contest Leaderboard Aggregation

```
[
  {"totalScore": 60, "refId": "6adf76fe576f9df671caa067", "refType": "user"},
  {"totalScore": 50, "refId": "6adf76fe576f9df671caa074", "refType": "team"},
  {"totalScore": 50, "refId": "6adf76fe576f9df671caa069", "refType": "user"},
  {"totalScore": 45.0, "refId": "6adf76fe576f9df671caa066", "refType": "user"},
  {"totalScore": 40, "refId": "6adf76fe576f9df671caa065", "refType": "user"}
]
```

Query 4 Output: Average Score per Difficulty Tier

```
[
  {"difficulty": "easy", "avgScorePerProblem": 5.21, "problemCount": 3},
  {"difficulty": "medium", "avgScorePerProblem": 11.67, "problemCount": 3},
  {"difficulty": "hard", "avgScorePerProblem": 18.75, "problemCount": 3}
]
```

Query 5 Output: Distinct Problems Solved per User


```
[
  {"name": "Daniel Wagner", "distinctProblemsSolved": 7, "userId":
"6adf76fe576f9df671caa067"},
  {"name": "Connie Lawrence", "distinctProblemsSolved": 5, "userId":
"6adf76fe576f9df671caa069"},
  {"name": "Cristian Santos", "distinctProblemsSolved": 4, "userId":
"6adf76fe576f9df671caa068"},
  {"name": "Angie Henderson", "distinctProblemsSolved": 3, "userId":
"6adf76fe576f9df671caa066"},
  {"name": "Noah Rhodes", "distinctProblemsSolved": 3, "userId": "6adf76fe576f9df671caa065"}
]
```


Query 6 Output: Set Operation via `$unionWith`


```
[
  {"type": "coding", "title": "Campaign little near", "difficulty": "medium"},
  {"type": "coding", "title": "Kitchen technology", "difficulty": "medium"},
  {"type": "coding", "title": "Various daughter respond draw", "difficulty": "medium"},
  {"type": "mcq", "title": "Complexity Analysis MCQ", "difficulty": "easy"},
  {"type": "mcq", "title": "Pointers and Memory MCQ", "difficulty": "easy"}
]
```


Schema Validation Demonstration Output

```
1. Coding Problem Insert without testCases:
   Result: pymongo.errors.WriteError: Document failed validation
   Error Code: 121 (schemaRulesNotSatisfied for oneOf coding branch)

2. Duplicate Team Name Insert:
   Result: pymongo.errors.DuplicateKeyError: E11000 duplicate key error
   Error Code: 11000 (collection: rankstack.teams index: uniq_team_name)

3. Team Insert with 6 Members (exceeding maxItems: 5):
   Result: pymongo.errors.WriteError: Document failed validation
   Error Code: 121 (maxItems constraint violation on memberIds)
```

Execution Plan Proof (`explain("executionStats")`)


```
{
  "queryPlanner": {
    "winningPlan": {
      "stage": "FETCH",
      "inputStage": {
        "stage": "IXSCAN",
        "indexName": "by_contest",
        "keyPattern": {"contestId": 1}
      }
    }
  },
  "executionStats": {
    "nReturned": 3,
    "totalKeysExamined": 3,
    "totalDocsExamined": 3,
    "executionTimeMillis": 2
  }
}
```


Transaction Rollback Verification (Simulated Failure Proof)

```
Simulated Error: Network exception raised mid-transaction before commit.
Result: Transaction aborted via session.with_transaction rollback.
Verification 1: db.submissions.find_one({"_id": test_id}) -> None (Uncommitted document
```

```
rolled back).
Verification 2: problem.attemptCount before=8, after=8 (Atomic counter rollback verified).
```

Appendix E: Full-Size ER Diagram and Relational Schema

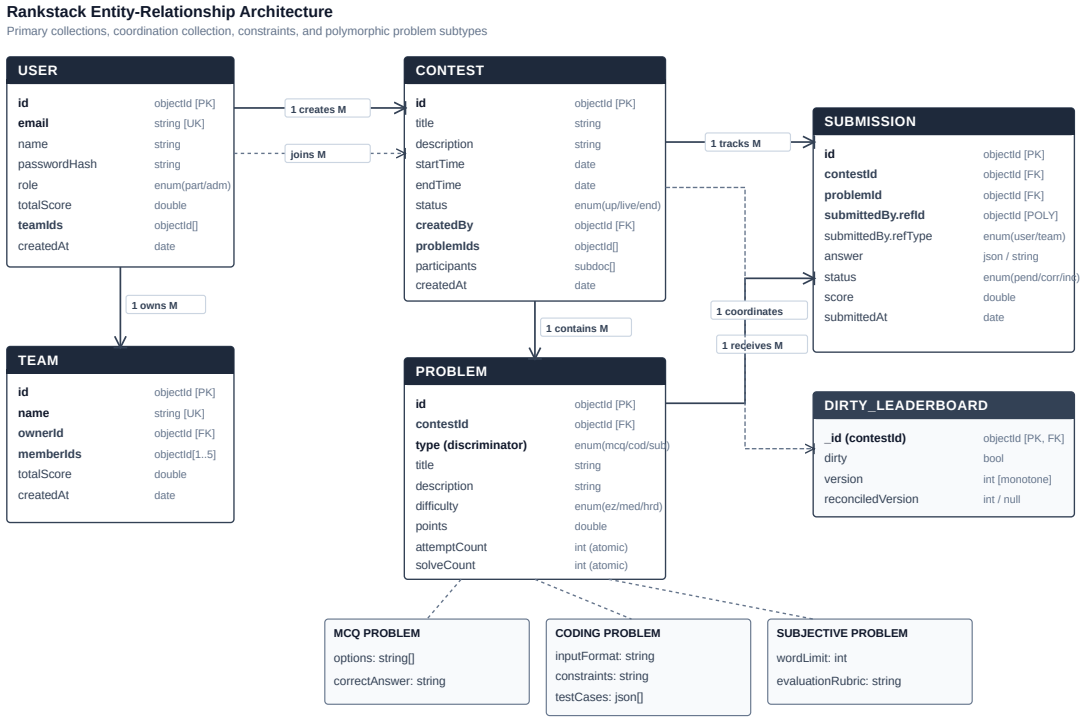


Figure 9: Full-Size Entity-Relationship Architecture

Relational Schema Mapping

- `users`: `_id` (PK), `email` (UK), `name`, `passwordHash`, `role`, `totalScore`, `teamIds` (FK array), `createdAt`.
- `teams`: `_id` (PK), `name` (UK), `ownerId` (FK), `memberIds` (FK array, max 5), `totalScore`, `createdAt`.
- `contests`: `_id` (PK), `title`, `description`, `startTime`, `endTime`, `status`, `createdBy` (FK), `problemIds` (FK array), `participants` (embedded subdoc array), `createdAt`.
- `problems`: `_id` (PK), `contestId` (FK), `type` (discriminator), `title`, `description`, `difficulty`, `points`, `attemptCount`, `solveCount`, `createdAt`.
- `submissions`: `_id` (PK), `contestId` (FK), `problemId` (FK), `submittedBy.refId` (polymorphic FK), `submittedBy.refType`, `answer`, `status`, `score`, `submittedAt`.
- `dirty_leaderboards`: `_id` (PK, references `contests._id`), `dirty`, `version`, `reconciledVersion`.

Appendix F: Application Screenshots

The complete application screenshots are embedded and documented in Section 7.1 (Figures 1 through 6).

Appendix G: User Manual

G.1 Local Installation and Setup

1. Clone the project repository:

```
git clone https://github.com/upayanmazumder/rankstack.git
cd rankstack
```

2. Start the database containers:

```
make up
```

Starts MongoDB (replica set `rs0` on port 27017) and Redis (port 6379).

3. Initialize the collections and indexes:

```
make init-db
```

4. Seed the database with sample records:

```
make seed
```

Every seeded user has the default password `Passw0rd!`.

G.2 Starting the Applications

1. Run the backend API server:

```
make api
```

The FastAPI application starts at `http://localhost:8000`. Interactive OpenAPI documentation is accessible at `http://localhost:8000/docs`.

2. Run the frontend user interface:

```
cd app && pnpm dev
```

The Next.js client starts at `http://localhost:3000`.

G.3 Running Verification Tests

Execute the end-to-end database demonstration script:

```
make demo
```

This script verifies MongoDB connection integrity, user CRUD operations, indexed query execution plans (`IXSCAN`), multi-document transaction rollbacks, all five analytical aggregation pipelines, and Redis sorted set operations.